

Python Functions

Mastering Code Reusability and Organization

```
def greet ( name ) :  
    # A simple function example  
    print ( f"Hello, {name}!" )
```

 Built-in

 User-defined

 Lambda

What is a Function?

Definition

- 📦 A **named, self-contained** block of code designed to perform a specific task
- ▶️ Only runs when it is **called**
- ↩️ Can optionally **return data** as a result

A simple function definition

```
def greet():  
    print("Hello from a function!")
```

Calling the function

```
greet() # Output: "Hello from a function!"
```

Types of Functions

📁 Built-in Functions

- | | |
|-----------|---------|
| ✓ print() | ✓ len() |
| ✓ type() | ✓ int() |

👤 User-defined Functions

Created by developers to address specific needs

```
def calculate_area(length, width):  
    return length * width
```

Why Use Functions?

Functions offer several key benefits for writing organized and efficient Python code:



Code Reusability

Write code once and execute it multiple times, adhering to the "Don't Repeat Yourself" (DRY) principle. Changes only need to be made in one place.



Modularity

Break down complex programs into smaller, manageable, and independent units, each performing a specific task.



Abstraction

Manage complexity by providing a simplified interface. Users only need to know the function name and expected inputs.



Maintainability

Code structured with functions is easier to understand, correct, enhance, and adapt over time. Changes can be isolated to specific functions.



Testability

Functions with clear arguments and explicit return values are easier to test, ensuring they behave as expected.



Functions are the building blocks of well-structured programs

Basic Function Syntax

</> Function Definition

Use the **def** keyword to define a function:

```
def function_name(parameter1,  
parameter2):  
    # Function body  
    pass
```

- Function name should be descriptive
- Parameters are placeholders
- Use meaningful parameter names

👉 Function Calling

To execute a function, call it by name:

```
# Calling a function  
function_name(argument1, argument2)
```

- Use function name followed by parentheses
- Provide arguments matching parameters
- Function executes when called

↔ Parameters vs Arguments

Parameters

Placeholders in function definition

```
def  
greet(name)  
:
```

Arguments

Actual values when calling

```
greet("Alice")
```

Example:

```
def greet(name):  
    print(f"Hello, {name}!")  
  
greet("Alice") # Output: Hello,  
Alice!
```

The Return Statement

⇌ Purpose

The `return` statement serves two primary purposes:

- Sends a value back to the caller
- Immediately terminates function execution

```
def add(a, b):  
    result = a + b  
    return result  
  
sum_result = add(3, 5)
```

Single Value

Functions can return any data type:

- ✓ Numbers
- ✓ Strings
- ✓ Lists, dictionaries, etc.

```
def square(number):  
    return number * number  
  
result = square(5)  
# Returns 25
```

📦 Multiple Values

Functions can return multiple values:

- ✓ Values are packed into a tuple
- ✓ Can be unpacked during assignment

```
def get_coordinates():  
    x = 10  
    y = 20  
    return x, y  
  
coords = get_coordinates()  
# Returns (10, 20)
```

👻 Implicit None

If no return statement is provided:

- ! Function implicitly returns `None`
- ! Equivalent to `return None`

```
def do_nothing():  
    pass  
  
result = do_nothing()  
# Returns None
```

Function Arguments Types

Feature	↓☰ Positional Arguments	🔑 Keyword Arguments
Order	Strict order matching parameters	Order does not matter; arguments are named
Readability	Can be less readable with many args	Enhances readability by naming parameters
Flexibility	Less flexible; changing order causes errors	More flexible; order can be arbitrary
Requirement	Required unless default value provided	Can be optional with default value

```
# Function definition
def calculate_cost(item, quantity,
price):
print(f"{quantity} {item} cost
${quantity * price:.2f}")
```

↓☰ Positional Arguments

```
# Order matters
# calculate_cost("bananas", 6,
0.74)
# Output: 6 bananas cost $4.44
```

❗ **Error:** Changing order causes bugs

🔑 Keyword Arguments

```
# Order does not matter
# calculate_cost(price=0.74,
quantity=6, item="bananas")
# Output: 6 bananas cost $4.44
```

✅ **Best practice:** Improves readability

Default Arguments

🔑 Using Default Arguments

- ✓ Parameters with predefined values if no argument is provided
- ✓ Makes parameters **optional** in function calls

```
# Function with default parameters
def greet(name="World", message="Hello"):
    print(f"{message}, {name}!")
```

```
greet() # Output: "Hello, World!"
```

```
greet("Alice") # Output: "Hello, Alice!"
```

```
greet("Bob", "Hi") # Output: "Hi, Bob!"
```

⚠️ Best Practices

Mutable Default Arguments

Avoid using mutable objects (lists, dictionaries) as default values:

```
# DON'T DO THIS
def add_item(item, items=[]): # Risky!
    items.append(item)
    return items
```

Recommended Approach

Use None as default and initialize inside function:

```
# DO THIS INSTEAD
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items
```

Arbitrary Arguments

☰ *args

Allows a function to accept an arbitrary number of **positional arguments**

- Arguments are collected into a **tuple**
- The ***** operator is the unpacking operator

```
def sum_all(*numbers):  
    return sum(numbers)  
  
result = sum_all(1, 2, 3, 4, 5)
```

Output: 15

🔑 **kwargs

Allows a function to accept an arbitrary number of **keyword arguments**

- Arguments are collected into a **dictionary**
- The ****** operator unpacks keyword arguments

```
def display_info(**details):  
    for key, value in details.items():  
        print(f"{key}: {value}")  
  
display_info(name="Alice", age=30)
```

Output:
name: Alice
age: 30

Lambda Functions

Definition & Syntax

- Small, **anonymous functions** defined with the **lambda** keyword
- Can take any number of **arguments**
- Can only have one **expression**
- Expression is **automatically returned**, no need for **return** statement

Lambda syntax

lambda arguments: expression

Example: Multiply three numbers

x = **lambda** a, b, c : a * b * c

print(x(5, 4, 3)) *# Output: 60*

Comparison & Use Cases

Equivalent regular function

def multiply(a, b, c):

return a * b * c

print(multiply(5, 4, 3)) *# Output: 60*

Common Use Cases

filter()

Filter elements based on a condition

map()

Apply a function to each element

sorted()

Sort using custom key function

Lambda with map() to square numbers

numbers = [1, 2, 3, 4, 5]

squared_numbers = **list**(**map**(**lambda** x: x ** 2, numbers))

print(squared_numbers) *# Output: [1, 4, 9, 16, 25]*

Decorators

What are Decorators?

- 🧩 Powerful feature that modifies function behavior **without altering original code**
- 📦 "Wrap" another function, adding functionality before/after execution
- @ Applied using @ syntax above function definition

```
# Original function
def my_function():
    pass
```



```
# Decorator
@my_decorator
def my_function():
```

Example: Logging Decorator

```
# Decorator definition
def log_function_call(func):
    def wrapper(*args, **kwargs):
        # Before function call
        print(f"Calling function: {func.__name__}")
        result = func(*args, **kwargs)
        # After function call
        print(f"Function {func.__name__} finished. Result: {result}")
    return wrapper
return wrapper
```

```
# Function with decorator
@log_function_call
def add(a, b):
    return a + b
```

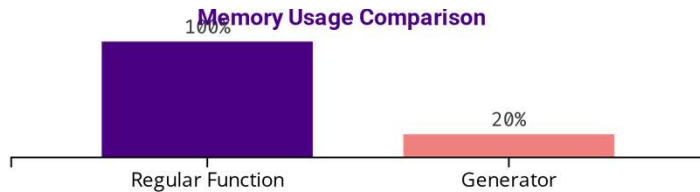
@log_function_call is syntactic sugar for `add = log_function_call(add)`

Generators

What are Generators?

- 🚂 Special type of **iterable** that produces values one at a time
- 🔌 Highly **memory-efficient** for large datasets
- ⏸ Function execution **pauses** at **yield** and resumes on next call
- 📄 Defined like regular functions but use **yield** instead of **return**

Memory Efficiency



Generator Example

```
# Generator function
def count_up_to(max_num):
    count = 1
    while count <= max_num:
        yield count
        count += 1
```

```
# Using the generator
counter = count_up_to(5)
# Generator is lazy - no execution yet!
for number in counter:
    print(number)
```

```
# Output:
# 1
# 2
# 3
# 4
# 5
```

Documentation Best Practices

Docstrings

-  String literals as the first statement in a function
-  Concise summary of purpose, arguments, and return values
-  Accessed via `__doc__` attribute or `help()` function

```
"""Calculate the average of a list of numbers.  
  
Parameters  
-----  
numbers : list[float]  
A list of numeric values.  
  
Returns  
-----  
float  
The arithmetic mean of the provided numbers.  
"""
```

Type Annotations

-  Optional specification of expected types (Python 3.5+)
-  Improves code clarity and enables static type checkers
-  Added with colons (:) for parameters and arrows (->) for return values

```
def calculate_average(numbers: list[float]) -> float:  
    # Function implementation  
    return sum(numbers) / len(numbers)
```

 **Best Practice: Combine docstrings and type annotations**

Real-World Applications



Data Analysis

Functions clean, transform, and analyze datasets

```
def calculate_stats(data):  
    return sum(data)/len(data)
```



Web Development

Functions process requests and return responses

```
def handle_request():  
    return render_template('index.html')
```



Automation

Functions automate repetitive tasks

```
def rename_files(pattern):  
    for file in os.listdir():  
        os.rename(file, file.replace(pattern,  
''))
```



Machine Learning

Functions define models and implement training loops

```
def train_model(data):  
    model = create_model()  
    for epoch in range(100):  
        model.fit(data)
```



Software Development

Functions create modular components for software systems

```
def create_gui():  
    button = create_button('Click me')  
    window.add_component(button)
```

💡 Python functions are the building blocks of efficient and maintainable code across diverse domains